10.2 Working with roles

Roles provide a simple mechanism for grouping together users who have the same level of access. They are most useful in applications that are unlikely to grow in complexity and where it is easy to differentiate the access needs of different groups of users. Identity includes support for roles, but it must be enabled by adding role-related services to the container with the `AddRoles<TRole>` method. `IdentityRole` is the default implementation representing a role:

```
builder.Services.AddDefaultIdentity<CityBreaksUser>(options => {
    options.Password.RequireDigit = false;
    options.Password.RequireLowercase = false;
    options.Password.RequireNonAlphanumeric = false;
    options.Password.RequireUppercase = false;
})
    .AddRoles<IdentityRole>()                          ❶
    .AddEntityFrameworkStores<CityBreaksContext>();
```

❶ Use the AddRoles method to add role-related services to the application.

There is no UI for managing roles, so you have to build your own. The primary API for working with roles is the `RoleManager<TRole>` service, which was one of the services added to the service container by the `AddRoles` method. The `RoleManager` includes the following CRUD methods:

- `CreateAsync`
- `UpdateAsync`
- `DeleteAsync`

The `RoleManager` also includes a property, `Roles`, which returns all of the roles from the `RoleStore`. The `RoleStore` represents the